

1. CSV Dataset with Nulls: bookings.csv

Create in Colab:

```
%writefile bookings.csv
booking_id,customer_name,city,service_type,provider,booking_amount,bookir
1001,Aarav Mehta,Hyderabad,Flight,IndiGo,6500,Confirmed,UPI
1002,Sana Khan,Bangalore,Hotel,Pearl Grand,4500,Confirmed,Card
1003,John Mathew,,Flight,Air India,12000,Confirmed,UPI
1004,Ayesha Begum,Hyderabad,Hotel,,7500,Pending,Cash
1005,Vikram Rao,Mumbai,Flight,Vistara,,Confirmed,Card
1006,Divya Sharma,Delhi,Flight,IndiGo,5900,Cancelled,
1007,Imran Ali,Pune,Hotel,Budget Inn,2200,,UPI
1008,Meera Nair,Kochi,Hotel,Hill View Resort,7500,Confirmed,Card
1009,Rohan Das,Kolkata,Flight,Air India,7400,Pending,UPI
1010,Nisha Reddy,Bangalore,Flight,British Airways,62000,Confirmed,Card
1011,Farhan Ali,,Hotel,Skyline Suites,22000,Confirmed,
1012,Neha Singh,Hyderabad,,Emirates,28000,Confirmed,UPI
1013,Arjun Verma,Chennai,Flight,,15000,Cancelled,Cash
1014,Kavya Nair,Mumbai,Hotel,Sea View Stay,,Pending,Card
1015,Ravi Kumar,Delhi,Flight,SpiceJet,4800,Confirmed,UPI
```

Exercises on CSV Null Handling

1. Read bookings.csv.
2. Display schema.
3. Count total records.
4. Find records where city is null.
5. Find records where provider is null.
6. Find records where booking_amount is null.
7. Find records where booking_status is null.

8. Find records where payment_mode is null.
9. Count null values in each column.
10. Drop all rows having any null value.
11. Drop rows where booking_amount is null.
12. Drop rows where customer_name, service_type or booking_amount is null
13. Fill null city with Unknown.
14. Fill null provider with Not Available.
15. Fill null payment_mode with Not Provided.
16. Fill null booking_status with Unknown.
17. Fill null booking_amount with 0.
18. Create data_quality_status column:
If any important field is null -> Incomplete
Else -> Complete
19. Count records by data_quality_status.
20. Display only Complete records.
21. Display only Incomplete records.
22. Create tax column = booking_amount * 5%.
23. Create final_amount = booking_amount + tax.
24. Calculate revenue only from confirmed bookings.
25. Count bookings by service_type after handling nulls.
26. Count bookings by city after filling missing cities.
27. Find average booking amount after replacing null amount with 0.
28. Find average booking amount after dropping null amount rows.

29. Compare both averages.

30. Save clean data as clean_bookings.parquet.

11. JSON Dataset with Nulls: customers.json

Create in Colab:

```
%%writefile customers.json
[
  {
    "customer_id": 1,
    "name": "Aarav Mehta",
    "city": "Hyderabad",
    "membership": "Gold",
    "contact": {
      "phone": "9876500011",
      "email": "aarav@mail.com"
    },
    "preferences": {
      "preferred_service": "Flight",
      "budget_range": "Medium"
    }
  },
  {
    "customer_id": 2,
    "name": "Sana Khan",
    "city": "Bangalore",
    "membership": "Silver",
    "contact": {
      "phone": null,
      "email": "sana@mail.com"
    },
    "preferences": {
      "preferred_service": "Hotel",
      "budget_range": null
    }
  },
  {
    "customer_id": 3,
```

```
[
  {
    "name": "John Mathew",
    "city": null,
    "membership": "Gold",
    "contact": {
      "phone": "9876500013",
      "email": null
    },
    "preferences": {
      "preferred_service": "Flight",
      "budget_range": "High"
    }
  },
  {
    "customer_id": 4,
    "name": "Ayesha Begum",
    "city": "Hyderabad",
    "membership": null,
    "contact": {
      "phone": "9876500014",
      "email": "ayesha@mail.com"
    },
    "preferences": {
      "preferred_service": null,
      "budget_range": "Low"
    }
  },
  {
    "customer_id": 5,
    "name": "Vikram Rao",
    "city": "Mumbai",
    "membership": "Platinum",
    "contact": {
      "phone": null,
      "email": null
    },
    "preferences": {
      "preferred_service": "Flight",
      "budget_range": "High"
    }
  }
]
```

Read JSON:

```
customers_df = spark.read.option(  
    "multiline",  
    "true"  
) .json("customers.json")  
  
customers_df.show(truncate=False)  
customers_df.printSchema()
```

12. Flatten JSON

```
flat_customers_df = customers_df.select(  
    "customer_id",  
    "name",  
    "city",  
    "membership",  
    col("contact.phone").alias("phone"),  
    col("contact.email").alias("email"),  
    col("preferences.preferred_service").alias("preferred_service"),  
    col("preferences.budget_range").alias("budget_range")  
)  
  
flat_customers_df.show()
```

JSON Exercises

1. Read customers.json.
2. Display schema.
3. Display all customer records.
4. Flatten contact.phone and contact.email.
5. Flatten preferences.preferred_service and preferences.budget_range.
6. Display customer name, city, phone and email.

7. Find customers where city is null.
8. Find customers where phone is null.
9. Find customers where email is null.
10. Find customers where membership is null.
11. Find customers where preferred_service is null.
12. Find customers where budget_range is null.
13. Count null values in flattened customer DataFrame.
14. Fill null city with Unknown.
15. Fill null membership with Standard.
16. Fill null phone with Not Provided.
17. Fill null email with Not Provided.
18. Fill null preferred_service with Not Selected.
19. Fill null budget_range with Unknown.
20. Create customer_quality_status:
 If city, phone, email, membership or preferred_service is null -> Inc
 Else -> Complete
21. Count customers by customer_quality_status.
22. Display only complete customers.
23. Display only incomplete customers.
24. Count customers by membership after handling nulls.
25. Count customers by preferred_service after handling nulls.
26. Save flattened customer data as customers_flat.parquet.
27. Save clean customer data as clean_customers.csv.

28. Compare original record count and clean record count.

29. Display customers with missing contact details.

30. Display customers with missing preference details.